

Dynamic Programming: Paint House

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 24, 2026

© 2026 Arpit Raj

Problem Setup

Suppose we have:

- House 0
- House 1
- House 2
- House 3

Each house can be painted **Red**, **Blue**, or **Green**. Each color has a different cost.

Example Costs

House	Red	Blue	Green
0	17	2	17
1	16	16	5
2	14	3	19

Goal:

- Adjacent houses cannot have same color.
- Minimize total cost.

State Definition

We'll define `dp[i][j]`

Meaning: Minimum cost to paint houses from 0...i such that house i is painted with color j.

where:

- `j = 0` → Red
- `j = 1` → Blue
- `j = 2` → Green

Base Case

```
dp[0][0] = cost[0][0];  
dp[0][1] = cost[0][1];  
dp[0][2] = cost[0][2]; // base case
```

State Transition

Suppose we are computing `dp[i][0]`

meaning: Current house is **Red**.

Previous house cannot be **Red**.

So previous must be **Blue** or **Green**.

Therefore:

```
dp[i][0] = cost[i][0] + min(dp[i-1][1], dp[i-1][2]);
```

Similarly:

```
dp[i][1] = cost[i][1] + min(dp[i-1][0], dp[i-1][2]);  
dp[i][2] = cost[i][2] + min(dp[i-1][0], dp[i-1][1]);
```

Code Implementation

```
class Solution {  
public:  
    int minCost(vector<vector<int>>& costs) {  
  
        int n = costs.size();  
        //dp[i][j] will define ith house kitna min cost me hua he  
        vector<vector<int>> dp(n, vector<int>(3, 0));  
  
        // Base case  
        // Cost of painting the first house  
        dp[0][0] = costs[0][0];  
        dp[0][1] = costs[0][1];  
        dp[0][2] = costs[0][2];  
  
        // Paint remaining houses  
        for(int i = 1; i < n; i++) {  
  
            // Paint current house Red  
            // Previous house can be Blue or Green
```

```
        dp[i][0] = costs[i][0] + min(dp[i-1][1], dp[i-1][2]);

        // Paint current house Blue
        // Previous house can be Red or Green
        dp[i][1] = costs[i][1] + min(dp[i-1][0], dp[i-1][2]);

        // Paint current house Green
        // Previous house can be Red or Blue
        dp[i][2] = costs[i][2] + min(dp[i-1][0], dp[i-1][1]);
    }

    // Last house can end with any color
    // We want the minimum total cost
    return min({dp[n-1][0], dp[n-1][1], dp[n-1][2]});
}

};
```